

Module 2

Topics: SAS dataset creation, Concatenating and merging datasets

Download the following files to an H drive folder before you begin: RAWDATA.TXT, RAWDAT1.xls, RAWDAT2.xlsx, phase1.sas7bdat, phase2.sas7bdat, labs.sas7bdat, cohort1516.sas7bdat, cohort1617.sas7bdat, survey.sas7bdat

Temporary vs Permanent SAS Datasets:

One of the most important things to understand about SAS datasets is that they can be temporary or permanent. A temporary SAS dataset is one that is created during a SAS session and then is automatically deleted when you exit the SAS program. On the other hand, a permanent SAS dataset is stored on a physical drive and can be used again in subsequent SAS sessions. You determine whether a SAS dataset is temporary or permanent by the way in which you name the dataset when you create it.

All SAS datasets have a two-level name with the two levels separated by a period. The first level is the library reference and tells SAS where the file is stored. SAS calls the library in which it stores temporary SAS datasets the WORK library and creates this library automatically at program start-up. If you want to create a permanent SAS dataset, you must create a library in which to store the file. You perform this task by using a LIBNAME statement. The LIBNAME statement associates a word of your choice to the physical drive on which you wish to store the permanent SAS dataset. An example of a LIBNAME statement is as follows:

```
libname NICOLE 'H:\AEPI537D';
```

This statement tells SAS that you wish to create a SAS library called NICOLE and this library resides in the AEPI537D folder on the H: drive of your computer. The following are examples of SAS dataset names and the type of datasets they represent:

<u>Dataset name</u>	<u>Type of dataset</u>	<u>Location of database file</u>
WORK.SURVEY	Temporary	Work library only during current session
SURVEY	Temporary	Work library only during current session
NICOLE.SURVEY	Permanent	SASDATA folder on H: drive

2.1 - Creating SAS Datasets

There are numerous ways to get data into a SAS dataset.

DATALINES

```
DATA LESSON1;
INPUT STUDYID GENDER $ HEIGHT WEIGHT HAIR $ EYE $;
DATALINES;
01 FEMALE 64 120 BROWN BROWN
02 MALE 70 180 BLOND BLUE
```

AEPI537D

```
03 MALE 72 200 BROWN BLUE
04 FEMALE 68 140 BROWN BROWN
05 MALE 71 170 BLOND BLUE
;
RUN;
```

The dollar sign after the variable names GENDER, HAIR and EYE tell SAS that these variables will store character data. The data are read with list input. This is the easiest type of input but it has some restrictions. The fields must be separated by at least one blank.

```
DATA LESSON2;
INFILE 'H:\AEPI537D\RAWDATA.TXT';
INPUT STUDYID CACO $ AGE EXPOSURE $;
RUN;
```

The INFILE statement tells SAS where the data are located.

Here is how the data appear in the dataset RAWDATA.TXT which is located on the H: drive in the AEPI537D folder:

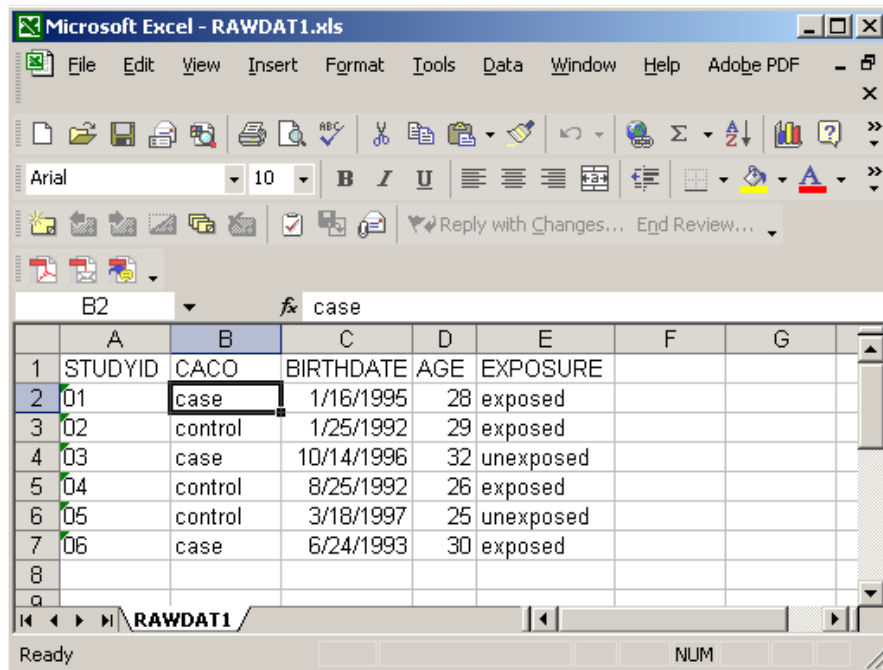
```
01 case 28 exposed
02 control 29 exposed
03 case 32 unexposed
04 control 26 exposed
05 control 25 unexposed
06 case 30 exposed
```

```
DATA LESSON3;
INPUT @1 STUDYID @6 CACO $ @14 DATE MMDDYY10.;
DATALINES;
1000 CONTROL 08/19/1997
1001 CASE 04/24/2000
1003 CONTROL 12/02/1998
1004 CASE 09/16/1998
;
RUN;
```

IMPORTING DATA INTO SAS

SAS has an import feature that allows you to translate files in various formats directly into a SAS dataset.

The file RAWDAT1.XLS contains data that have been entered into a Microsoft Excel database. Let's look at the dataset in Excel first. Notice that the first row of the spreadsheet contains the variable names. Also notice that the data for BIRTHDATE is in date format.

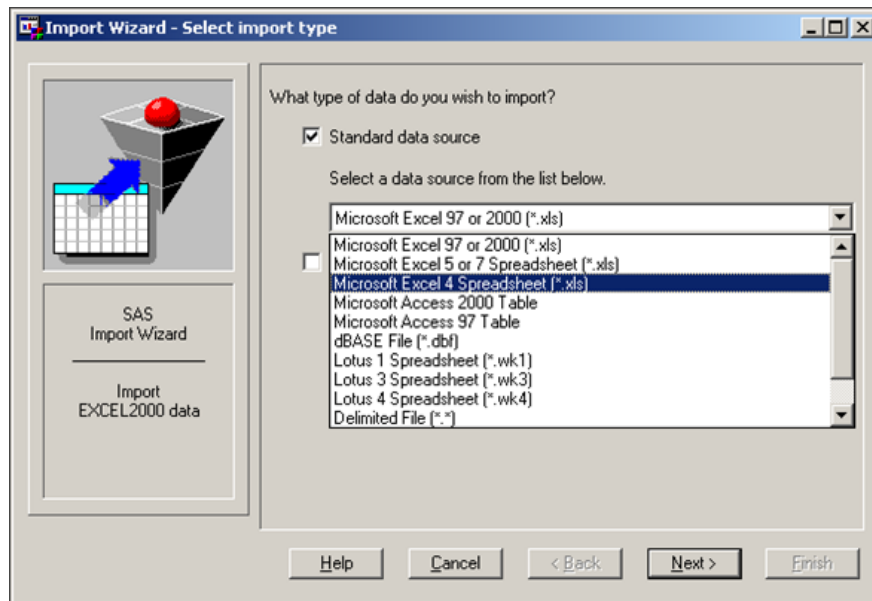


	A	B	C	D	E	F	G
1	STUDYID	CACO	BIRTHDATE	AGE	EXPOSURE		
2	01	case	1/16/1995	28	exposed		
3	02	control	1/25/1992	29	exposed		
4	03	case	10/14/1996	32	unexposed		
5	04	control	8/25/1992	26	exposed		
6	05	control	3/18/1997	25	unexposed		
7	06	case	6/24/1993	30	exposed		
8							
9							

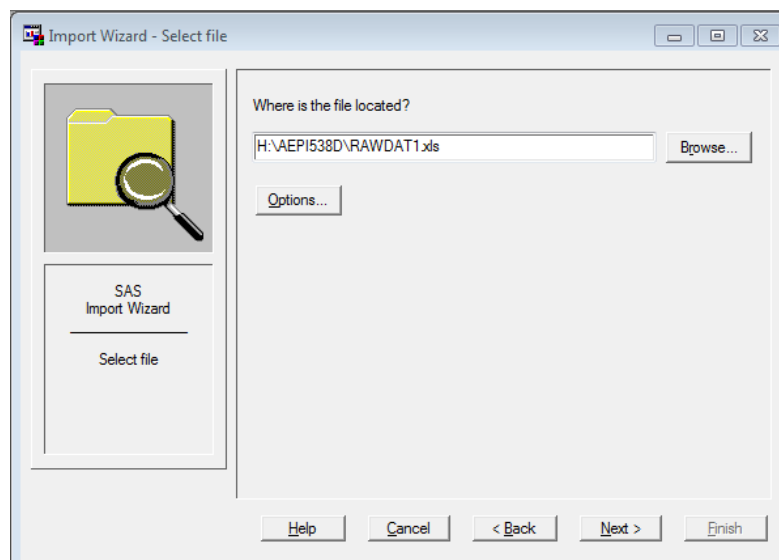
The following steps will demonstrate the use of the SAS Import Wizard to translate (import) an Excel file into a SAS file.

.XLS (Excel 4.0) -->> SAS (Using Import Wizard)

1. Open SAS
2. Submit LIBNAME NICOLE 'H:\AEPI537D'
3. Click on FILE in the SAS Menu
4. Click on IMPORT DATA.
5. Select File Format "Microsoft Excel 4 Spreadsheet (*.xls)" from the drop down list in the standard data source section of the menu, then Click on NEXT.



6. Type in filename and location (H:\AEPI537D\RAWDAT1.xls). Use BROWSE to find the file if you are unsure of the filename, then Click NEXT.



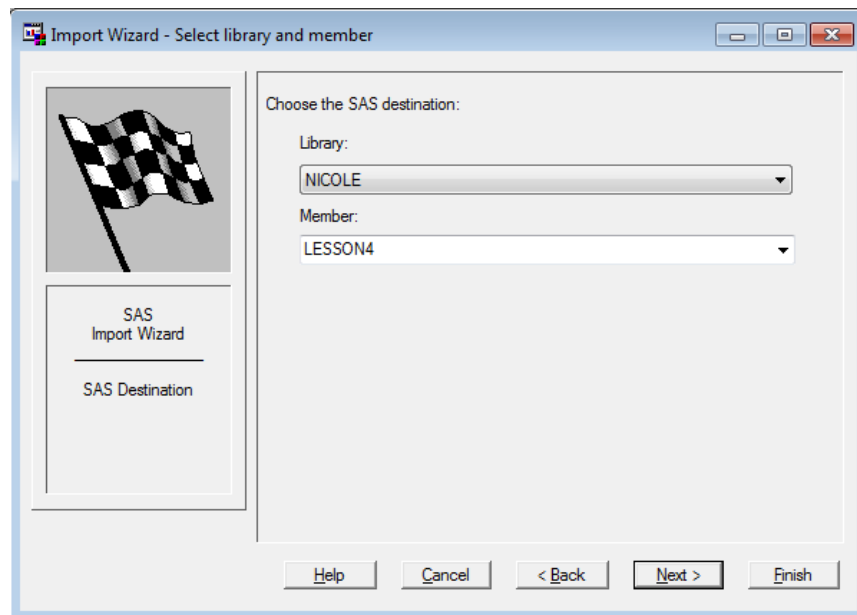
7. Choose the SAS destination (this means the name and location of the SAS Dataset that you are creating)

Select Library

if the file you wish to create is to be a temporary SAS dataset, select WORK

if the file is to be a permanent SAS dataset, select the LIBNAME from Step 2

Type in Member (the name you wish to give the SAS dataset, e.g., LESSON4)



8. Click on FINISH

To make sure that the import was completed, check the SAS log. You should see the line “NICOLE.LESSON4 was successfully created” if everything went as planned.

You can also create import steps in your SAS code to avoid using the import wizard. This is helpful when writing a program that will be used several times. SAS has specific import syntax for most file types.

Here is one way to import the XLSX file RAWDAT2 and create a SAS dataset called LESSON5. The GETNAMES=YES setting tells SAS to use the first row of data as variable names.

```
PROC IMPORT OUT= WORK.LESSON5 DATAFILE= "H:\AEPI537D\RAWDAT2.xlsx"
  DBMS=xlsx REPLACE;
  SHEET="Sheet1";
  GETNAMES=YES;
RUN;
```

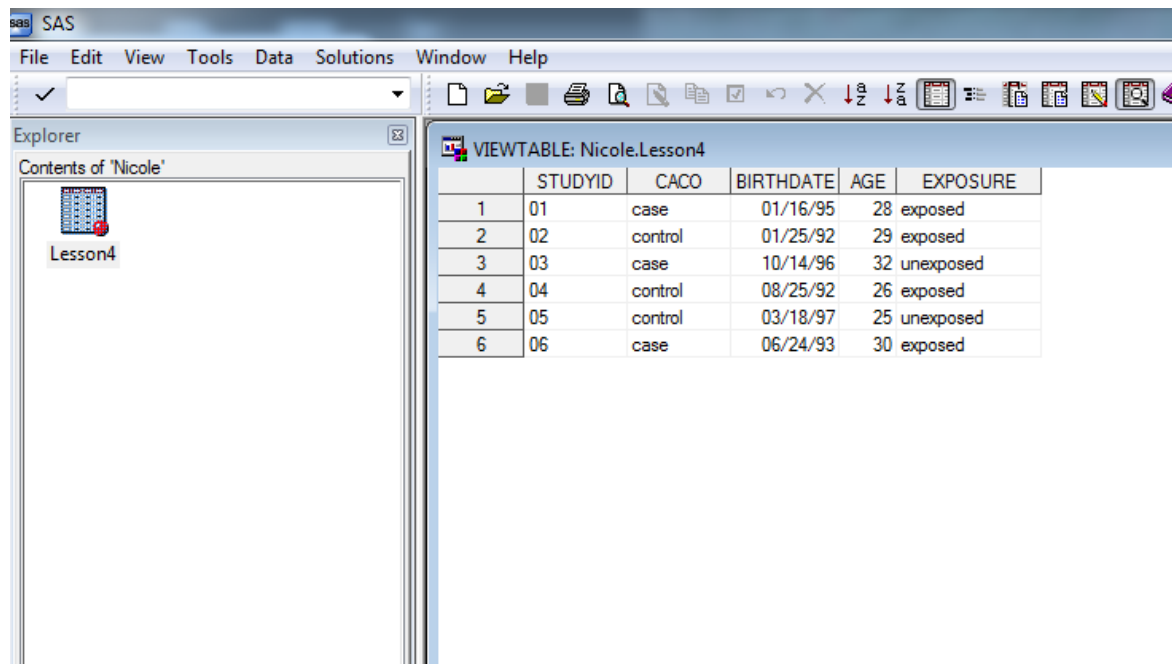
2.2 - Using features of SAS Explorer

The SAS Explorer gives you a number of ways to work with your SAS datasets. You can view a dataset, and examine the variables and their attributes using Explorer.

Let's take a look at the permanent SAS dataset that we created in the last section. We created a SAS dataset called LESSON4 in the AEPI537D folder on the H: drive. Submit a LIBNAME statement to SAS that identifies the drive.

```
libname NICOLE 'H:\AEPI537D';
```

Go to the SAS Explorer and double click on the LIBRARIES icon. Now double click on the NICOLE library. You should now see an icon called LESSON4 which is our SAS dataset. Double click on the LESSON4 icon and a browser window will open that contains a spreadsheet view of your SAS dataset.

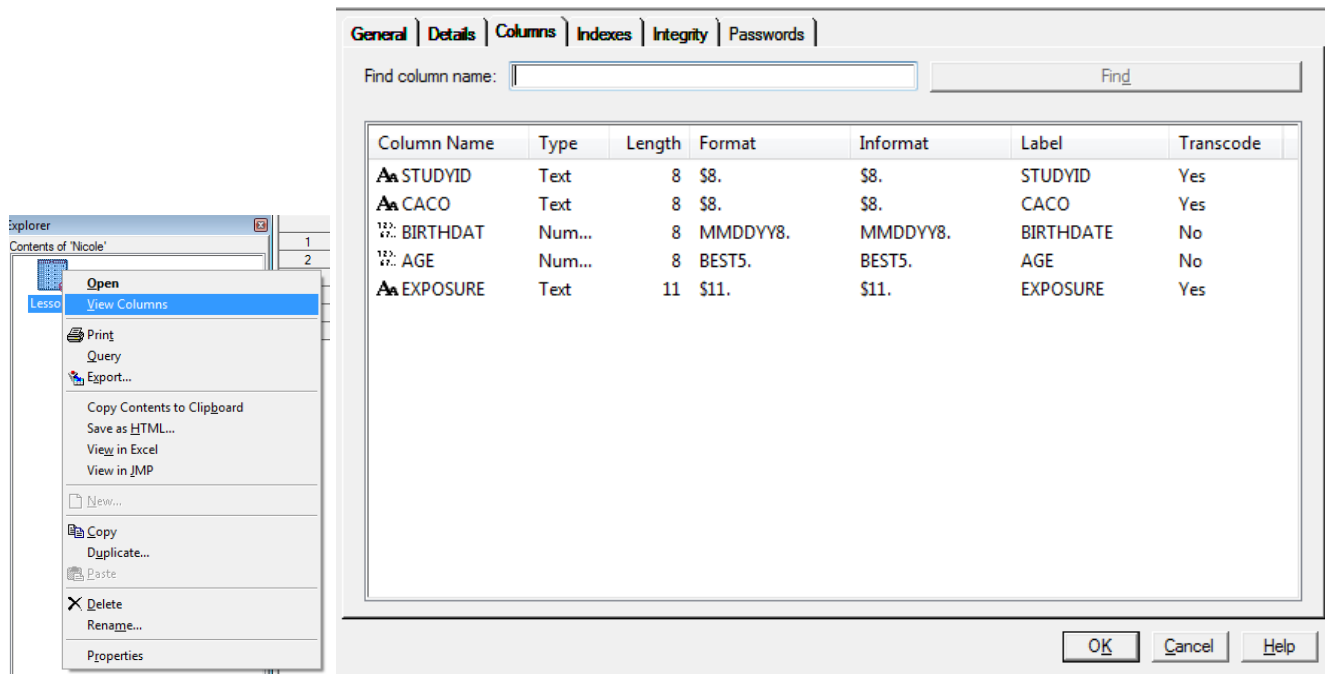


	STUDYID	CACO	BIRTHDATE	AGE	EXPOSURE
1	01	case	01/16/95	28	exposed
2	02	control	01/25/92	29	exposed
3	03	case	10/14/96	32	unexposed
4	04	control	08/25/92	26	exposed
5	05	control	03/18/97	25	unexposed
6	06	case	06/24/93	30	exposed

SAS calls this a VIEWTABLE. You can use this view to scan through your dataset. By default, SAS opens the table in browse mode, which means that you can only view your data. Be sure to close the VIEWTABLE before returning to the Editor. If you have a VIEWTABLE view open and you try to run a program on the dataset you may get an error message in the SAS Log telling you that the dataset is unavailable because it is in use by another program.

You can get a complete listing of all of the dataset variables and their attributes using the CONTENTS procedure in SAS. You can also accomplish this by using the “View columns” option in the SAS Explorer. Place the cursor over the LESSON4 icon in the Explorer window and click the RIGHT mouse button. This will bring up a menu that includes “View Columns” as the second choice.

Left click once on View Columns to select it. This opens another window that SAS calls the “Properties” window. This window contains a listing of all of the variables in the dataset in the order that they appear in the dataset accompanied by their attributes.



2.3 - Some useful procedures in SAS

There are several very useful procedures that let you become familiar with the data stored in SAS datasets. The key SAS word for a procedure is PROC. We will cover four essential PROCs here: PROC CONTENTS, PROC PRINT, PROC FREQ and PROC UNIVARIATE.

You previously downloaded three SAS datasets from Canvas called PHASE1, PHASE2, and LABS and copied them to your network drive. Let's get familiar with these files using these procedures.

PROC CONTENTS DATA=NICOLE.PHASE1;
RUN;

This procedure gives you essential information about the SAS dataset including when it was created, how many observations and variables it contains and a listing of all variables and their attributes.

PHASE1 contains data on sex, age, and body mass index (BMI) for 20 patients who were enrolled in the first phase of a genetic study. Each person was assigned a unique studyid.

PROC CONTENTS varnum DATA=NICOLE.PHASE2;
RUN;

By adding the VARNUM option to the CONTENTS procedure the variables will appear according to their order in the dataset (the default is to show them alphabetically).

PHASE2 contains the same information (same variables) on 10 more patients who were enrolled in the study during a later phase.

```
PROC CONTENTS DATA=NICOLE.LABS;  
RUN;
```

LABS is a file that contains the result of genetic testing for all 30 of the patients enrolled in both phases of the study.

We can use a print procedure to learn about the values of the variables stored in the SAS datasets. For example, if we wanted to see the values stored in the variables STUDYID, SEX and AGE in the PHASE1 dataset we would use the following code.

```
PROC PRINT DATA=NICOLE.PHASE1;  
    VAR STUDYID SEX AGE;  
RUN;
```

This procedure sends a line listing of the values stored in each variable listed on the VAR line for each observation in the dataset. If you do not specify the variables using the VAR statement, you will get a listing for all of the variables in the dataset. PROC PRINT has a variety of options you can use to customize the output. NOOBS and LABEL are very useful. The example below is the same print procedure we just ran, but it's using NOOBS to hide the observation numbers and LABEL to add some additional detail to the column headers.

```
PROC PRINT DATA=NICOLE.PHASE1 NOOBS LABEL;  
    VAR STUDYID SEX AGE;  
    LABEL STUDYID= 'Participant ID'  
          SEX= 'Sex at birth'  
          AGE= 'Age' ;  
RUN;
```

Now let's take a look at the FREQ procedure.

```
PROC FREQ DATA=NICOLE.PHASE2;  
    TABLES SEX AGE;  
RUN;
```

This procedure provides summary statistical information about each of the variables in the TABLES line including the number of observations that have each value stored in the variable along with proportions and other statistics. This procedure is most useful for variables that contain categorical data – that is, data that has a limited number of possible values such as male and female, mild or severe, true or false, etc.

If you are interested in summary statistics on continuous data, the UNIVARIATE procedure is more useful than the FREQ procedure.

```
PROC UNIVARIATE DATA=NICOLE.PHASE2;
```



```
VAR AGE;  
RUN;
```

This procedure is similar to the FREQ procedure except it is most useful for obtaining summary statistics on continuous data – that is, data that have a potentially unlimited number of values such as weight, BMI, years of age, etc.

2.4 - Concatenating and Merging data files

Now we are going to learn two very important techniques for combining data from separate files. This is a very common task that you may be required to perform. For example, demographic and clinical patient data are often stored in one database while the results of lab tests and x-rays, etc. are stored in another database. In order to analyze all of the data you must know how to combine data from more than one dataset.

There are two main ways that datasets can be combined - concatenation and merging. Concatenation is the process by which two datasets with the same variables (data) but containing different observations are combined. You will recall that observations (people in your study) are kept in the rows of the database. Therefore, when two datasets are concatenated, the observations in one dataset are added to the observations in the other. The concatenated dataset will have all of the observations that were contained in each of the smaller datasets. Because the variables are the same in both datasets, the concatenated dataset will have the same number of variables as in each of the smaller datasets.

Example: Concatenation adds the observations of one data set to another.

<u>Data Set One</u>	+	<u>Data Set Two</u>	=	<u>Data Set Three</u> (Concatenation of One and Two)
01 Var1		05 Var1		01 Var1
02 Var1		06 Var1		02 Var1
03 Var1		07 Var1		03 Var1
04 Var1		08 Var1		04 Var1
				05 Var1
				06 Var1
				07 Var1
				08 Var1

Merging, on the other hand, is the process by which two datasets with the same observations (people) but different variables (data) are combined. In this case, we have two or more datasets that contain different data (variables) about the same group of people (observations). The merged dataset will contain the same number of observations as each of the component datasets but will contain all of the variables that were in both datasets.

Example: Merging adds the variables of one data set to another.

<u>Data Set One</u>	+	<u>Data Set Two</u>	=	<u>Data Set Three</u>	(Merge of One and Two)
01 Var1		01 Var2		01 Var1 Var2	
02 Var1		02 Var2		02 Var1 Var2	
03 Var1		03 Var2		03 Var1 Var2	
04 Var1		04 Var2		04 Var1 Var2	

Concatenation is less complicated than merging because we are adding separate observations from a second database that are independent from those of the first database. However, merging data variables from two datasets requires that the datasets have a common variable (e.g., STUDYID) that will allow the variables from the second database to be added to the correct observation in the first database. The following example will demonstrate a real-life situation that will require you to combine the data from multiple files using concatenation and merge.

We will use the data stored in the SAS datasets PHASE1, PHASE2 and LABS that we just became familiar with using our SAS procedures. Our task is to create one SAS dataset that contains all of the patients in both phases and also includes the results of the genetic test for each patient.

In the first step, we will combine all of the observations from the phase1 and phase2 studies into one database. Because both files contain the same variables and we wish to combine observations, we will use the concatenate procedure. The following steps will concatenate the PHASE1 and PHASE2 SAS files into a temporary SAS dataset called PHASE12.

```
DATA PHASE12;
    SET NICOLE.PHASE1 NICOLE.PHASE2;
RUN;
```

This program starts with a data step in order to create a new temporary SAS dataset called PHASE12 that will contain all of the observations in the PHASE1 and PHASE2 SAS datasets. Because these are SAS datasets we must refer to them using a SET statement. What is unusual about this SET statement is that we refer to **two** SAS datasets. When SAS reads this, it interprets the statement as: concatenate PHASE1 and PHASE2 into a new SAS dataset called PHASE12. Let's take a look at the resulting dataset:

```
PROC PRINT DATA = PHASE12;
RUN;
```

You can see that we have now completed the first part of our task. The concatenation combined all of the clinical data collected on patients who were enrolled in both phases of the study. All 30 observations are contained in the PHASE12 SAS dataset.

Now let's take a look at the LABS dataset.

```
PROC PRINT DATA = NICOLE.LABS;
RUN;
```

Notice that both datasets have a common variable - STUDYID. If you look at the studyids in each database you will notice that they are not in any kind of order. How are we going to get the lab data from the LABS database into the PHASE12 database making sure that each observation gets the correct lab value?

Here is where we can use the other method of combining datasets - the MERGE. In order to merge two or more datasets, the datasets that are to be merged MUST contain a variable in common. Usually, this is some sort of identification number or maybe the person's name. The problem with using names to identify subjects in a study (aside from the confidentiality issue) is that some names are very common and it is possible that you could have more than one person in your dataset with the same name.

Our datasets have the common variable STUDYID. In order to merge the datasets and ensure that the correct lab result gets assigned to each person, the datasets must first be sorted on the common variable. The following steps perform the required data sorting:

```
PROC SORT DATA = PHASE12;  
  BY STUDYID;  
RUN;
```

```
PROC SORT DATA = NICOLE.LABS;  
  BY STUDYID;  
RUN;
```

Now we are ready to perform the MERGE. The following statements will create a new permanent SAS dataset called LESSON6 that will contain the merged data:

```
DATA NICOLE.LESSON6;  
  MERGE PHASE12 NICOLE.LABS;  
  by studyid;  
RUN;
```

Now let's take a look at the resulting dataset:

```
PROC PRINT DATA = NICOLE.LESSON6;  
RUN;
```

Notice that all 30 observations are in the dataset and that the studyids have been put in order from lowest to highest. In addition, the new dataset contains all of the variables from both datasets. Each person was assured of getting his correct lab result because the data were brought together (merged) according to the STUDYID.